MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF REPUBLIC OF MOLDOVA
TECHNICAL UNIVERSITY OF MOLDOVA
FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE ENGINEERING AND AUTOMATICS

CodeCraft: learning programming through Minecraft. Project report

Mentor: Crucerescu Vladislav
Students: Chiril Boboc, FAF-242
Vasile Brînză, FAF-242
Cristian Bruma, FAF-242
Gabriela Bîtca, FAF-242
Teodor Strulea, FAF-242

Chișinău, 2025

Abstract

According to our standards, the first paragraph of the abstract needs to contain the following:

- Name of the project,
- Author/-s,
- Institution,
- Report structure.

Keywords: LaTeX, TeX, Research Paper, Project.

A typical abstract is a 150 to 250 word paragraph that provides readers with a quick overview of the essay or report and the way it is organized. It should express the central idea of the project and the key points. It should also mention possible applications of the research being discussed in the paper.

Some of the things that can be included in the abstract are:

- The purpose of the project,
- The general objectives of it,
- Used methodologies,
- The obtained results.

Content

Introduction **4**

1 Domain Analysis **5**

1.1 Educational Context and Learning Objectives 5

1.1.1 Educational programming tools for children 5

1.1.2 Gamification of learning programming 5

1.1.3 Educational DSL Precedents and Benefits 5

1.2 Problem Analysis 6

1.3 Solution Proposal 7

1.3.1 Specific Problems the DSL Solves 8

1.3.2 Use Case Scenarios 9

1.4 Target User Research 10

1.4.1 Age Ranges and Skill Levels of Target Users 10

1.4.2 Accessibility Requirements and Learning Challenges 10

1.4.3 Influence of the DSL on Adjacent Users 10

1.4.4 Stakeholders Involved 10

2 Language Design **12**

2.1 DSL Requirements 12

2.1.1 General Language Rules 12

2.1.2 Language Paradigm 12

2.1.3 Core Language Constructs 12

2.1.4 Abstraction Level and Complexity of Functionalities 13

2.1.5 DSL Limitations 13

2.2 The Grammar 13

2.2.1 Grammar Notation Conventions 14

2.3 Lexer Implementation 14

2.3.1 Token Recognition 14

2.3.2 Indentation Handling 14

2.3.3 Keyword Resolution and Special Cases 15

2.4 Parser Implementation 15

Conclusions **16**

Bibliography **17**

A BNF Grammar **22**

Introduction

The purpose of the introduction is to inform the reader/evaluator about the content and the general objectives of the paper. In this part of the paper, the following aspects should be covered:

- Studied domain,
- Motivation for choosing the topic,
- Degree of novelty / relevance of the topic or of the problem,
- General objectives of the project,
- Research methodologies that have been used,
- A brief sneak peek to the content of the subsequent chapters.

The volume of the introduction usually is about 2-3 pages.

1 Domain Analysis

1.1 Educational Context and Learning Objectives

In today's tech-driven world, there's a growing need to develop skills for navigating the digital era. Computational thinking (CT) has become vital, fostering analytical and problem-solving skills crucial for coding proficiency [1]. Integrating programming skills into education can enhance CT abilities, that are essential for future readiness, particularly among primary school students [2].

1.1.1 Educational programming tools for children

Typically, children begin learning programming at the stage of middle school. One of the tools they are taught with, is Scratch, a popular simplified block-based language. Scratch's aesthetic aspects are the most significant component for developing computational thinking [3]. Research indicates that game-based programming environments like Scratch serve as crucial factors for improving education quality by changing student attitudes toward the frustrations of learning programming logic and fostering skill development through experimentation in motivating contexts [4]. Other effective tools for teaching programming are Kodable, CodeMonkey, BlocklyGames. Their main effectiveness in developing computational thinking comes from their gamified design, which engages children by providing immediate feedback and having a memorable interface [5].

1.1.2 Gamification of learning programming

The integration of game elements into the learning process can contribute to the development of algorithmic thinking, logic and creative abilities of students from an early age [6]. Game methods of teaching programming differ from others in that the game is a natural learning environment that attracts students, increases their motivation and stimulates active research, allows students to apply theoretical knowledge in practice, receiving immediate feedback and observing the results of their work, promotes the development of not only technical programming skills, but also such important qualities as creativity, critical thinking and problem-solving skills, and is an interactive process that allows students to interact with educational material in a dynamic and exciting environment, creates a center for cooperation and exchange of experience between students, contributing to the development of team skills [7]. The perceived enjoyment and social influence yielded by gamified programming instruction have a high impact on student's motivation to learn programming [8].

1.1.3 Educational DSL Precedents and Benefits

Logo represents one of the earliest educational DSLs, designed in 1967 to teach programming concepts through turtle graphics and LISP-like constructs [9]. Game Maker Language similarly combines simplified commands derived from GPLs (General Purpose Languages) to enable users to create games

without advanced programming knowledge [9]. Well-designed DSLs improve programmer productivity, addressing key challenges in software development [10].

1.2 Problem Analysis

Modern children face severe attention span challenges that make traditional programming education ineffective. Standard programming courses require prolonged focus, abstract thinking, and patience for delayed results. Interface complexity becomes an obstacle instead of supporting the learning process. These requirements clash with how children consume information today.

Speaking of existing tools, there are technical barriers. Children face Java requirements far above beginner level when working with Minecraft mods. Tools like ComputerCraft use Lua syntax that confuses young learners. Setup complexity takes time before anyone writes a single line of code, which stops learning early. Command blocks, an in-game programming interface, overwhelm beginners from the start. Parents or teachers must handle installation and configuration because children lack the technical knowledge.

As for pedagogical gaps, children learn abstract concepts without seeing corresponding actions in the game. Block-based tools hide text syntax and create barriers when transitioning to real code. Motivation drops when results feel distant or unclear. Available tools do not match how children interact with digital products today. Educational resources assume adult knowledge or prior programming experience.

Pain Points in Existing Solutions for Teaching Programming:

- **Scratch and Other Visual Languages:** Scratch lowers the entry barrier by removing syntax errors, but it operates in an abstract environment disconnected from the games children already play. The visual block system hides real text-based programming syntax, which makes the transition to textual languages like Python or Java difficult [11]. As projects grow in complexity, block-based coding limits the types of programs learners can build [12]. Learners often struggle to connect their work to familiar gaming environments, causing engagement to drop [13].
- **Minecraft Education Edition:** Minecraft Education Edition integrates learning into a familiar game, but access requires institutional licensing, limiting home use [14]. The platform targets classroom instruction and emphasizes guided lessons over open-ended experimentation [15]. Activities rely heavily on pre-built curricula, which reduces creative freedom and limits exploration. Many children prefer the regular version of Minecraft where friends play, making the educational edition feel isolated from their social gaming environment [16].
- **ComputerCraft:** ComputerCraft introduces real programming concepts through Lua, but it requires text-based coding from the start. Installation and setup demand technical knowledge often unavailable to parents. Documentation assumes prior programming experience, which can overwhelm beginners. Young learners face high complexity before achieving results, leading to frustration and early disengagement [17].

- **Other Common Approaches and Their Limitations:** Minecraft command blocks allow in-game automation, but dense syntax produces frequent errors that confuse beginners. Redstone teaches logic indirectly, and debugging circuits is unclear, making reasoning about failures difficult. Traditional programming languages such as Python or Java require setup and configuration before learners can see results; syntax errors in early stages often block progress and reduce motivation.

Thus, the problem statement is the following: **Children disengage from programming education when learning tools fail to provide immediate visual feedback and do not connect abstract coding concepts to activities that sustain their interest.**

1.3 Solution Proposal

Young learners need a programming environment where code produces instant visual results within a game they love. Minecraft offers a solution because children already invest hours in the game and want to improve their gameplay, additionally, it also offers an interactive digital environment that closely aligns with core programming and computational thinking concepts. Actions performed by players occur in a clear sequence, which reflects the execution of instructions in imperative programming. Repetitive gameplay activities such as mining blocks, farming resources, or crafting items correspond to loop structures, where the same operation is repeated until a specific condition is satisfied. Decision-making in response to in-game situations, including reacting to nightfall, selecting appropriate tools, or avoiding hostile entities, mirrors conditional logic used in programming. Managing inventory items is similar to dealing with variables that change over time, while recurring building patterns and construction methods resemble the use of procedures and modular design. In-game events such as mob spawning or environmental changes parallel event-driven programming models. These conceptual parallels have been identified and discussed in multiple studies examining the educational use of Minecraft in STEM and computational thinking contexts [18, 19].

A study focusing on Minecraft-based coding activities with middle school students reported statistically significant improvements in computational thinking dimensions such as sequencing, loops, conditionals, events, and parallelism following a structured instructional intervention [20]. Systematic reviews of empirical studies from the past decade also find that Minecraft can help learners grasp basic programming concepts, provided that learning goals and instructional support are clearly defined [19, 18].

Minecraft has seen substantial adoption in formal educational contexts, primarily through Minecraft Education Edition, which is designed specifically for classroom use. According to Microsoft Education, tens of millions of teachers and students across more than one hundred countries have access to licensed versions of the platform, indicating widespread institutional uptake rather than isolated experimentation [14]. A systematic literature review covering studies published between 2015 and 2024 confirms that Minecraft has been implemented across primary and secondary education, most frequently within STEM subjects,

and that many educators continue to use it beyond initial trials [18]. Although precise global percentages of schools using Minecraft are not consistently reported, the scale of adoption and the volume of peer-reviewed research demonstrate that Minecraft is an established educational platform.

Research on the educational effectiveness of Minecraft consistently reports increased student engagement, motivation, and participation when the platform is integrated into structured learning activities. Studies grounded in constructivist and social constructivist learning theories highlight that Minecraft supports inquiry-based learning, problem solving, collaboration, and spatial reasoning [21, 22]. Experimental research further indicates that Minecraft-based interventions can lead to measurable improvements in spatial and cognitive skills, provided that instructional goals are explicit and teacher guidance is present [23].

At the same time, the literature identifies significant limitations related to technical and programming complexity. Multiple studies report that educators often struggle to design effective Minecraft-based lessons due to limited programming knowledge and lack of confidence with the platform's technical features [24, 19]. A recent systematic review emphasizes that insufficient teacher training and high technical barriers frequently result in Minecraft being used in a superficial manner, which reduces its potential to support deeper learning outcomes [18]. These findings indicate that Minecraft's educational benefits are strongly dependent on pedagogical structure and accessible tooling.

This gap creates an opportunity for a domain-specific language that turns programming into an engaging experience. Therefore, the proposed solution is the development of a domain specific language that is suited for children to script actions in Minecraft, with the purpose of learning programming.

1.3.1 Specific Problems the DSL Solves

- **Lack of Immediate Visual Feedback:** Children write code and expect to see action happen in the game world. Existing tools separate code execution from visible results. Learning slows when outcomes stay invisible or appear only as text in a console. The DSL runs commands directly inside the game environment. Children watch their bot perform actions the moment they execute the script. Blocks appear, items move, and structures are built in real time while the code runs.
- **Weak Path Toward Real Programming:** Children learn logic through drag-and-drop blocks, then face text-based code later without preparation. The transition feels abrupt and confusing. The DSL uses simple text commands from the beginning. Children read and write actual code syntax instead of clicking visual blocks. The syntax remains clear and readable while teaching real programming structure. When children move to other languages later, they already understand how text-based code works.
- **Excessive Technical Complexity:** Current tools require deep technical knowledge to accomplish basic tasks. Setting up mods, understanding game mechanics, and navigating complex interfaces create a steep learning curve. The DSL hides these complex mechanics behind clear, simple commands.

Children focus on programming logic and problem structure instead of fighting technical barriers. Commands abstract away implementation details while still teaching fundamental concepts.

- **Poor Alignment with Modern Learning Patterns:** Traditional programming education uses long tutorials with delayed gratification. Children lose interest before seeing meaningful results. The DSL embeds learning directly inside Minecraft, where children already spend time. Each command triggers a visible change in the game world. Engagement stays high because every action produces immediate outcomes that children care about.
- **Missing Support for Independent Learning:** Children depend on adult help to progress through existing programming tools. Exploration stops when guidance becomes unavailable. The DSL supports independent trial and error through clear syntax and fast feedback loops. Children experiment with commands, see what happens, fix mistakes when things go wrong, and advance on their own. Parents without programming knowledge no longer need to intervene constantly.
- **Lack of Meaningful Context:** Programming exercises in traditional curricula feel arbitrary and disconnected from student interests. Children solve problems they do not care about, so motivation fades quickly. The DSL ties every coding task to goals children set for themselves in Minecraft. Building a house, automating a farm, or organizing inventory all serve purposes that children choose. Each line of code accomplishes something personally meaningful.

1.3.2 Use Case Scenarios

- **Build a House:** The goal is for the user to place blocks in a consistent pattern to form a simple structure. A sequence of movement and placement commands is written to define where each block goes. Correct results depend on executing commands in the right order, and repeating similar actions helps reinforce the idea of structured sequences and repetition.
- **Create a Farming Bot:** The objective is to plant crops in a straight line efficiently. A repeat instruction combines movement and planting actions so the same steps are applied across multiple blocks. This use case introduces loops and demonstrates how automation reduces manual effort.
- **Manage Inventory:** The goal is to remove or move items from specific inventory slots. Inventory and drop commands are written with parameters such as slot numbers and coordinates. This use case teaches how structured commands and parameters control precise behavior.
- **React to Conditions:** The objective is for the character to respond to changes in game state, such as eating when hunger drops below a certain level. A condition is defined with an associated action. This use case introduces decision-making and conditional logic.
- **Chest Management Bot:** The goal is to organize collected items into storage chests by type. A bot is programmed to move through a base, identify items, and place them into the correct chests. This use case combines movement, logic, and automation, reinforcing data organization and control flow

through a practical task.

1.4 Target User Research

1.4.1 Age Ranges and Skill Levels of Target Users

The DSL's target users are early adolescents, approximately 10–15 years of age, who are mature enough to read and type but are still novices in programming. The age ranges were inspired from studying existing programs; for example, the new Irish curricula introduce coding in "Junior Cycle" (approx. 12–15 years of age) [25]. These users have usually mastered basic computational thinking (often via visual tools) and are ready to tackle syntax-based tasks. The DSL being text-based implies that the learners must have sufficient typing skills, although it assumes no prior familiarity with programming syntax, as it is designed for learning. The DSL is aimed at primary-school and middle-school aged children transitioning from block-based environments toward text, who can type steadily and are building their first independent coding skills [25, 26].

1.4.2 Accessibility Requirements and Learning Challenges

Accessible design must accommodate young learners. From a learning standpoint, novices universally face conceptual hurdles. Reviews find that beginners often lack familiarity with syntax, algorithms, and debugging strategies, as they struggle with syntactic, conceptual, and strategic knowledge in introductory programming [27]. Common errors include forgetting punctuation or misunderstanding loops. The DSL must thus minimize unnecessary complexity through clear error messages and gradual feature introduction, and build on children's math and problem-solving skills, since those are known challenges in learning to code [27, 28].

1.4.3 Influence of the DSL on Adjacent Users

The choice of a text-based DSL affects not only the target user but also those around them. Text-based code influences novices to rely on the teacher for syntax help, so instructors become more involved in technical guidance [29]. The DSL shapes classroom culture; an intuitive, playful coding environment makes the class friendly and encourages peer interaction [29]. In practice, this means that if the DSL is engaging, siblings or classmates might be more inclined to join in or learn together. Likewise, parents may take on roles as helpers or reviewers of the child's code, and their experience will be influenced by how approachable the text language is. In sum, the DSL influences educators' teaching style and peers' collaboration.

1.4.4 Stakeholders Involved

Besides the children, multiple stakeholders are involved in the solution. Key educational stakeholders include teachers, school administrators, and parents [30], since they guide and support the child's learning process. Curriculum designers and educational researchers are also stakeholders, as they determine how a new language fits into learning standards and lesson plans [31]. In practice, administrators and

policymakers decide whether programming is part of the school curriculum, directly affecting the DSL's adoption. In summary, the ecosystem includes the student coders, their families, their teachers, the school leaders, and the broader education community — curriculum experts, policymakers, and developers — all of whom play roles in the design, deployment, and success of the DSL [30, 31].

2 Language Design

2.1 DSL Requirements

2.1.1 General Language Rules

Research indicates that children consistently struggle with strict syntax requirements, punctuation rules, language formalities, verbose statements, and inadequate debugging feedback [32]. Python addresses these challenges through its minimal syntax and readable structure, which explains its widespread adoption in introductory programming courses across universities worldwide [32]. These findings informed the decision to adopt Python as the primary source of inspiration for the DSL's syntax and overall design.

Empirical research on identifier naming conventions demonstrates that different styles — camelCase, PascalCase, and snake_case — impose measurably different levels of visual effort and affect the speed at which identifiers are recognised by the reader. Studies using eye-tracking methodology found that snake_case yields the lowest visual effort and the highest recognition speed, particularly among novice programmers [33]. Consequently, snake_case was selected as the identifier style for all predefined variables and functions in the DSL.

2.1.2 Language Paradigm

Given that the DSL targets children with minimal programming experience, an imperative paradigm was selected as the foundational approach. Empirical research demonstrates that imperative programming exhibits one of the lowest learning curves among major paradigms [34]. This paradigm is particularly suitable for novice learners, as it aligns with sequential, step-by-step reasoning patterns and facilitates code comprehension during debugging processes.

While object-oriented programming presents comparable learning difficulty to imperative programming [34], it was deemed less appropriate for this context. The object-oriented paradigm emphasizes abstraction and code encapsulation, concepts that may prove challenging for the target demographic. Additionally, object-oriented approaches typically require more verbose code structures, introducing unnecessary complexity beyond the DSL's intended scope.

2.1.3 Core Language Constructs

The DSL should include variables to familiarize children with a core component of most programming languages. Variables are essential for storing coordinates, object IDs, and other Minecraft elements that users need to manipulate.

To enable process automation, the DSL will incorporate conditional statements and iterative constructs for handling repetitive tasks. As these represent fundamental components of most programming languages, their inclusion supports progressive learning objectives.

2.1.4 Abstraction Level and Complexity of Functionalities

To preserve user flexibility and support progressive skill development, the DSL will adopt a layered abstraction approach, providing both primitive and composite operations. Primitive operations include atomic actions such as block placement, directional orientation, jumping, clicking, and inventory management. Composite operations encompass higher-level tasks including wall construction, water removal from defined areas, and automated crop cultivation.

This architectural design enables users to construct custom high-level functionalities by composing primitive operations when predefined composite operations do not satisfy specific requirements. Such an approach promotes understanding of functional composition in computational thinking.

2.1.5 DSL Limitations

To prevent implementation complexity and breaking Minecraft official policies, the DSL will not change the game's state with means external to the user's in-game character. Thus, the DSL allows the user to interact with the game only through their character.

Although the usage of the DSL is intended for children, the installation process of the DSL requires external assistance.

2.2 The Grammar

The DSL follows an imperative, statement-oriented structure where a program consists of a sequential list of statements. Each statement occupies its own line, with newlines acting as statement terminators rather than semicolons, reducing syntactic noise for novice learners.

The language supports six categories of statements: variable assignments, two forms of iteration, a while loop, a conditional, function calls, and single-line comments. Comments are introduced with a double dash (`--`), a deliberate choice to avoid conflict with the subtraction operator and to remain readable.

Iteration is provided in two forms. A count-based loop (`for N times`) allows repetition without requiring the learner to manage an explicit iterator, which lowers the cognitive barrier for simple repetitive tasks. A range-based loop (`for x from A to B`) introduces the concept of an iterator variable progressively, once the learner is ready for it. Both forms use indentation-based block delimitation, consistent with Python-like conventions that children are likely to encounter in subsequent languages.

Expressions follow a standard left-recursive arithmetic structure supporting addition, subtraction, multiplication, and division, with parentheses available for grouping. A special dot-notation construct (`object.property`) is provided to represent Minecraft-specific entities and their attributes, offering a lightweight form of structured data access without introducing full object-oriented semantics.

Identifiers follow conventional alphanumeric rules with underscore support. Literals include integers, quoted strings, and boolean values (`true`/`false`). Whitespace handling is made explicit in the

grammar, with optional spacing permitted around operators and delimiters to give learners flexibility in formatting their code.

2.2.1 Grammar Notation Conventions

The grammar is written in Backus–Naur Form (BNF), where non-terminal symbols are enclosed in angle brackets and terminal symbols appear as quoted string literals. Several special tokens and conventions are used throughout the grammar and are defined as follows:

- **NEWLINE:** Represents a line break character (`\n`), used to terminate statements and signal the end of a logical line.
- **INDENT:** Represents an increase in indentation level at the beginning of a line. The grammar does not prescribe a fixed indentation width; any consistent combination of spaces or tabs is accepted, consistent with learner-friendly formatting conventions.
- **DEDENT:** Represents a decrease in indentation level, signaling the end of an indented block. It is the logical counterpart of **INDENT**.
- **ε:** Denotes the empty string, used in productions where a symbol may derive nothing, such as optional whitespace or an empty character sequence.

The complete formal grammar in BNF notation is provided in Appendix A.

2.3 Lexer Implementation

The lexer is responsible for transforming raw source text into a flat sequence of tokens, which the parser then consumes. It operates line by line, handling both the structural indentation logic and the inline tokenization of each line's content.

2.3.1 Token Recognition

Token recognition is driven by a list of ordered token specifications, each pairing a `TokenType` with a regular expression pattern. The order is significant — more specific or longer patterns are listed before shorter ones to prevent incorrect partial matches. For example, the two-character operators `!=`, `==`, `<=`, and `>=` are specified before their single-character counterparts `<` and `>`.

All individual patterns are combined at class initialization into a single master regular expression using alternation, and compiled once into a `Pattern` object. During tokenization, a `Matcher` is advanced through the content of each line using `lookingAt()`, which attempts to match at the current position without requiring a full-string match. The capturing group that produced the match is then used to identify the corresponding `TokenSpec`.

2.3.2 Indentation Handling

Since the DSL uses indentation-based block delimitation, the lexer is responsible for emitting `INDENT` and `DEDENT` tokens explicitly. An integer stack tracks the indentation levels encountered so far, initialized with a base level of zero. At the start of each line, the number of leading spaces and tabs is counted. If the

measured indentation exceeds the top of the stack, an `INDENT` token is emitted and the new level is pushed. If it is less, `DEDENT` tokens are emitted and levels are popped until the stack matches the current indentation. Blank and whitespace-only lines are skipped entirely and carry no structural meaning.

2.3.3 Keyword Resolution and Special Cases

Identifiers are first matched by a general alphanumeric pattern, then passed through a keyword lookup to determine whether the matched text corresponds to a reserved word such as `for`, `while`, `if`, or `true`. If a keyword match is found, the corresponding keyword token type is emitted instead of a generic `IDENT` token.

String literals have their surrounding quotation marks stripped during tokenization, so the parser receives only the inner content. Comments, introduced by a double dash (`--`), cause the lexer to discard the remainder of the line immediately upon detection. Any character that does not match any known pattern results in an `ILLEGAL` token carrying the unrecognized character, allowing the parser to produce a meaningful error message rather than failing silently.

Each line concludes with an explicit `NEWLINE` token. At the end of the source, any remaining open indentation levels are closed with corresponding `DEDENT` tokens, followed by a final `EOF` token.

2.4 Parser Implementation

The parser consumes the token sequence produced by the lexer and constructs an Abstract Syntax Tree (AST). It is implemented as a hand-written recursive descent parser, where each grammar rule corresponds directly to a parsing method.

The parser maintains a cursor over the token list, advancing through it with `expect()`, which asserts the current token type and throws a `ParseException` with the line and column number if the assertion fails.

When the current token is an identifier, a one-token lookahead disambiguates between an assignment and a function call statement. The two forms of `for` loop are similarly disambiguated by checking whether the identifier after `for` is followed by the `from` keyword.

Expression parsing uses a two-level recursive descent encoding operator precedence: `parseExpression()` handles additive operators, `parseTerm()` handles multiplicative operators, and `parseFactor()` handles the base cases. Conditions are parsed separately, supporting negation, comparison operators, and bare identifiers or function calls used as boolean predicates.

All syntax errors are reported as a `ParseException` carrying the offending token, its literal value, and its position in the source.

Conclusions

Here go your conclusions..

Bibliography

[1] Tsai, M.-J.; Wang, C.-Y.; et al. Developing the Computer Programming Self-Efficacy Scale for Computer Literacy Education. *Journal of Educational Computing Research*, volume 56, no. 8, Jan. 2019: pp. 1345–1360, ISSN 0735-6331, 1541-4140, doi:10.1177/0735633117746747. Available from: `https://journals.sagepub.com/doi/10.1177/0735633117746747`

[2] Popat, S.; Starkey, L. Learning to code or coding to learn? A systematic review. *Computers & Education*, volume 128, Jan. 2019: pp. 365–376, ISSN 03601315, doi:10.1016/j.compedu.2018.10.005. Available from: `https://linkinghub.elsevier.com/retrieve/pii/S0360131518302768`

[3] Oscco Solorzano, R.; Torres Matos, L.; et al. Analysis of the Scratch program for computational thinking in schoolchildren in an educational institution in Peru: a network analysis. *Journal of Educational Technology Development and Exchange*, volume 18, no. 3, 2025: pp. 118–132, ISSN 19418035, doi: 10.18785/jetde.1803.06. Available from: `https://aquila.usm.edu/jetde/vol18/iss3/6/`

[4] Kuz, A. Computational thinking: an analysis through structured programming using Scratch. *Revista de Ciencia y Tecnología*, , no. 39, May 2023: pp. 82–90, ISSN 18517587, 03298922, doi:10.36995/j.recyt.2023.39.010. Available from: `https://www.fceqyn.unam.edu.ar/recyt/index.php/recyt/article/view/736`

[5] Hlazova, V.; Monoharova, V.; et al. Integration of game elements and digital tools into modern methods of programming teaching. *E-learning teXnology*, volume 8, Dec. 2024: pp. 24–29, ISSN 2709-8400, doi:10.31865/2709-840082024316932. Available from: `https://journals.uran.ua/texel/article/view/316932`

[6] Medvedieva, M.; Zhmurko, O.; et al. Use of online game services in the study of programming languages. *Humanities science current issues*, volume 2, no. 36, 2021: pp. 248–255, ISSN 23084855, doi:10.24919/2308-4863/36-2-40. Available from: `http://www.aphn-journal.in.ua/archive/36_2021/part_2/42.pdf`

[7] Moshkova, N.; Aleksieieva, H.; et al. Gamification as One of the Trends of Modern Education. *Molod i rynok*, , no. 4/224, May 2024: pp. 82–87, ISSN 2617-0825, 2308-4634, doi:10.24919/2308-4634.2024.300147. Available from: `http://mir.dspu.edu.ua/article/view/300147`

[8] Faculty of Education in Jagodina, University of Kragujevac, Serbia; Milutinović, V. R.; et al. EXPLORING PRIMARY SCHOOL STUDENTS' MOTIVATION AND ENJOYMENT IN LEARNING PROGRAMMING THROUGH GAMIFICATION. In *STEM/STEAM/STREAM APPROACH IN THEORY AND PRACTICE OF CONTEMPORARY EDUCATION : Conference Proceedings*, University of Kragujevac, Faculty of Education in Jagodina, 2025, ISBN 978-86-7604-242-5, pp. 231–244,

doi:10.46793/STREAM25.231M. Available from: https://doi.ub.kg.ac.rs/doi/zbornici/10-46793-stream25-231m/?

[9] Domain-Specific Language | Computer Science | Research Starters | EBSCO Research. Available from: https://www.ebsco.com

[10] Fowler, M. *Domain-specific languages*. The Addison-Wesley signature series, Upper Saddle River, N.J: Addison-Wesley, 2011, ISBN 978-0-321-71294-3 978-0-13-210754-9.

[11] Connecting Blocks to Create Scripts in Scratch Programming Language - PiEmbSysTech - Embedded Systems & VLSI Lab. July 2024, section: Programming Languages. Available from: https://piembsystech.com/connecting-blocks-to-create-scripts-in-scratch-programming-language/

[12] Guide, C. Scratch vs. Other Coding Platforms: An In-depth Comparison. Oct. 2023. Available from: https://learncodingusa.com/scratch-vs-other-coding-platforms/

[13] Adler, F.; Fraser, G.; et al. Improving Readability of Scratch Programs with Search-based Refactoring.

[14] Licensing. Available from: https://education.minecraft.net/en-us/licensing

[15] Pros and Cons of Using Minecraft Education in the Classroom. Jan. 2025. Available from: https://www.schoolsthatlead.org/blog/2025/2/25/the-pros-and-cons-of-using-minecraft-education-in-the-classroom

[16] Sripan, T.; Manyam, K. Gamified Learning: Teaching Coding and Creative Thinking with Minecraft: Education Edition (M:EE) for Thai Students. *Journal of Education and Learning*, volume 14, no. 4, Mar. 2025: p. 270, ISSN 1927-5269, 1927-5250, doi:10.5539/jel.v14n4p270. Available from: https://ccsenet.org/journal/index.php/jel/article/view/0/51458

[17] Kazemitabaar, M.; Chyhir, V.; et al. Scaffolding Progress: How Structured Editors Shape Novice Errors When Transitioning from Blocks to Text. 2023, doi:10.48550/ARXIV.2302.05708, version Number: 1. Available from: https://arxiv.org/abs/2302.05708

[18] Md Shamsudin, N.; Dalim, S. F.; et al. Crafting STEM Lessons: A Systematic Literature Review on the Use of Minecraft in Education (2015-2024). *International Journal of Academic Research in Business and Social Sciences*, volume 14, no. 10, Oct. 2024: pp. Pages 2820–2827, ISSN 2222-6990, doi:10.6007/IJARBSS/v14-i10/23231. Available from: https://hrmars.com/journals/papers/IJARBSS/v14-i10/23231

[19] Nebel, S.; Schneider, S.; et al. Mining Learning and Crafting Scientific Experiments: A Literature Review on the Use of Minecraft in Education and Research. *Educational Technology & Society*, volume 19, Jan. 2016: pp. 355–366.

[20] Tablatin, C. L. S.; Casano, J. D. L.; et al. Using Minecraft to cultivate student interest in STEM. *Frontiers in Education*, volume 8, Mar. 2023: p. 1127984, ISSN 2504-284X, doi:10.3389/feduc.2023.

1127984. Available from: https://www.frontiersin.org/articles/10.3389/feduc.2023.1127984/full

[21] Short, D. Teaching Scientific Concepts using a Virtual World - Minecraft. *Teaching Science*, volume 58, Sept. 2012: pp. 55–58.

[22] Kotsopoulos, D.; Floyd, L.; et al. A Pedagogical Framework for Computational Thinking. *Digital Experiences in Mathematics Education*, volume 3, no. 2, Aug. 2017: pp. 154–171, ISSN 2199-3246, 2199-3254, doi:10.1007/s40751-017-0031-2. Available from: http://link.springer.com/10.1007/s40751-017-0031-2

[23] Ming, G. K.; Sirisena, A. Exploring the Impact of Minecraft-Based Projects on STEM and SDG Integration in Education: A Case Study. *International Journal of Research and Innovation in Social Science*, volume IX, no. IIIS, 2025: pp. 5242–5252, ISSN 24546186, doi:10.47772/IJRISS.2025.903SEDU0378. Available from: https://rsisinternational.org/journals/ijriss/articles/exploring-the-impact-of-minecraft-based-projects-on-stem-and-sdg-integration-in-edu

[24] Bower, M.; Wood, L.; et al. Improving the Computational Thinking Pedagogical Capabilities of School Teachers. *Australian Journal of Teacher Education*, volume 42, no. 3, Jan. 2017, ISSN 1835-517X, doi:10.14221/ajte.2017v42n3.4. Available from: https://ro.ecu.edu.au/ajte/vol42/iss3/4

[25] Noone, M.; Mooney, A. Visual and textual programming languages: a systematic review of the literature. *Journal of Computers in Education*, volume 5, no. 2, June 2018: pp. 149–174, ISSN 2197-9987, 2197-9995, doi:10.1007/s40692-018-0101-5. Available from: http://link.springer.com/10.1007/s40692-018-0101-5

[26] Saito, D.; Washizaki, H.; et al. Comparison of Text-Based and Visual-Based Programming Input Methods for First-Time Learners. *Journal of Information Technology Education: Research*, volume 16, Jan. 2017: pp. 209–226, doi:10.28945/3775.

[27] Neuwinger, M.; Riehle, D. A Systematic Review of Common Beginner Programming Mistakes in Data Engineering. Apr. 2025, doi:10.48550/arXiv.2504.16644, arXiv:2504.16644 [cs]. Available from: http://arxiv.org/abs/2504.16644

[28] Nadeem, A.; Rubaab, J.; et al. Enhancing Computational Thinking of Children with Dyslexia using Visual Programming Languages Model. *Journal of Computer Science & Computational Mathematics*, volume 11, no. 3, Sept. 2021: pp. 69–78, ISSN 22318879, doi:10.20967/jcscm.2021.03.007. Available from: https://www.jcscm.net/cms/?action=showpaper&id=2205238

[29] Weintrop, D.; Wilensky, U. Comparing Block-Based and Text-Based Programming in High School Computer Science Classrooms. *ACM Transactions on Computing Education*, volume 18, no. 1, Mar. 2018: pp. 1–25, ISSN 1946-6226, doi:10.1145/3089799. Available from: https://dl.acm.org/

doi/10.1145/3089799

[30] Akkaya, S.; Erkan, A. Examining Stakeholder Opinions on Coding Education in Primary Schools. *International Journal of Contemporary Educational Research*, volume 12, no. 1, Mar. 2025: pp. 1–25, ISSN 2148-3868, doi:10.52380/ijcer.2025.12.1.568. Available from: https://ijcer.net/index.php/pub/article/view/568

[31] Stakeholders Involved in BPC. July 2022. Available from: http://ecepalliance.org/resources/models/stakeholders-involved-bpc/

[32] Klimeková, E. Is Python an Appropriate Programming Language for Teaching Programming in Secondary Schools? *International Journal of Information and Communication Technologies in Education*, volume 4, May 2015, doi:10.1515/ijicte-2015-0005.

[33] Sharif, B.; Maletic, J. *An Eye Tracking Study on camelCase and under_score Identifier Styles*. Aug. 2010, doi:10.1109/ICPC.2010.41, journal Abbreviation: IEEE International Conference on Program Comprehension Pages: 205 Publication Title: IEEE International Conference on Program Comprehension.

[34] Ugwumba, N. *Programming Paradigm Learning Curves: An Empirical Study of Developer Skill Acquisition in Functional, Object-Oriented, and Imperative Programming*. Jan. 2026, doi:10.13140/RG.2.2.26177.01129.

[35] Fowler, M. DSL Guide. Aug. 2019. Available from: https://martinfowler.com/dsl.html

[36] Tomassetti, F. The complete guide to (external) Domain Specific Languages. Feb. 2017. Available from: https://tomassetti.me/domain-specific-languages/

[37] Guzdial, M. Software-Realized Scaffolding to Facilitate Programming for Science Learning. *Interactive Learning Environments*, volume 4, no. 1, Jan. 1994: pp. 001–044, ISSN 1049-4820, 1744-5191, doi:10.1080/1049482940040101. Available from: http://www.tandfonline.com/doi/abs/10.1080/1049482940040101

[38] Gu, P.; Wu, J.; et al. Scaffolding self-regulation in project-based programming learning through online collaborative diaries to promote computational thinking. *Education and Information Technologies*, volume 30, no. 12, Aug. 2025: pp. 16243–16267, ISSN 1360-2357, 1573-7608, doi:10.1007/s10639-025-13367-1. Available from: https://link.springer.com/10.1007/s10639-025-13367-1

[39] Ma, B.; Li, H.; et al. Scaffolding Metacognition in Programming Education: Understanding Student-AI Interactions and Design Implications. 2025, doi:10.48550/ARXIV.2511.04144, version Number: 1. Available from: https://arxiv.org/abs/2511.04144

[40] Belland, B. R.; Kim, C.; et al. A Framework for Designing Scaffolds That Improve Motivation and Cognition. *Educational Psychologist*, volume 48, no. 4, Oct. 2013: pp. 243–270, ISSN 0046-1520,

1532-6985, doi:10.1080/00461520.2013.838920. Available from: http://www.tandfonline.com/doi/abs/10.1080/00461520.2013.838920

[41] Repenning, A.; Grabowski, S. Scaffolding Creative Programming Projects. In *Proceedings of the 19th WiPSCE Conference on Primary and Secondary Computing Education Research*, Munich Germany: ACM, Sept. 2024, ISBN 979-8-4007-1005-6, pp. 1–6, doi:10.1145/3677619.3677634. Available from: https://dl.acm.org/doi/10.1145/3677619.3677634

[42] Qian, Y.; Lehman, J. Students' Misconceptions and Other Difficulties in Introductory Programming: A Literature Review. *ACM Transactions on Computing Education*, volume 18, no. 1, Mar. 2018: pp. 1–24, ISSN 1946-6226, doi:10.1145/3077618. Available from: https://dl.acm.org/doi/10.1145/3077618

[43] Microsoft Computer Science Curriculum Whitepaper. Available from: https://www.microsoft.com/education

[44] Vol.14, No.4 | Journal of Education and Learning | CCSE. Available from: https://ccsenet.org/journal/index.php/jel/issue/view/0/3200

[45] Stalla, A. DSL for Modding Minecraft. Feb. 2024. Available from: https://tomassetti.me/dsl-for-minecraft/

[46] Minescript – Python scripting for Minecraft. Available from: https://minescript.net/

[47] SpigotDSL. Available from: https://www.spigotmc.org/resources/spigotdsl.79383/

[48] Enea, A. *Closing the Block-to-Text Gap: A Domain-Specific JavaScript Editor for Early Computational Thinking*. Oct. 2025, doi:10.48550/arXiv.2512.00012.

[49] Hermans, F. Hedy: A Gradual Language for Programming Education. In *Proceedings of the 2020 ACM Conference on International Computing Education Research*, ICER '20, New York, NY, USA: Association for Computing Machinery, 2020, ISBN 978-1-4503-7092-9, pp. 259–270, doi:10.1145/3372782.3406262. Available from: https://dl.acm.org/doi/10.1145/3372782.3406262

[50] Stefik, A.; Siebert, S. An Empirical Investigation into Programming Language Syntax. *ACM Transactions on Computing Education*, volume 13, no. 4, Nov. 2013: pp. 1–40, ISSN 1946-6226, doi:10.1145/2534973. Available from: https://dl.acm.org/doi/10.1145/2534973

[51] Volina, N.; Hlavacs, H. *Enhancing Programming learnability for Children through Video Games*, volume 19. Sept. 2025, doi:10.34190/ecgbl.19.2.3981, journal Abbreviation: European Conference on Games Based Learning Publication Title: European Conference on Games Based Learning.

A BNF Grammar

```
<program>           ::= <statement_list>

<statement_list>    ::= <statement>
                      | <statement> <statement_list>

<statement>         ::= <assignment> NEWLINE
                      | <for_count>
                      | <for_range>
                      | <while_loop>
                      | <if_stmt>
                      | "stop" NEWLINE
                      | <function_call> NEWLINE
                      | <comment>

<assignment>        ::= <identifier> <optional_space> "=" <optional_space>
    <expression>

<for_count>         ::= "for" <space> <expression> <space> "times" NEWLINE
    <block>
<for_range>         ::= "for" <space> <identifier> <space> "from" <space>
   <expression> <space> "to" <space> <expression> NEWLINE <block>
<while_loop>        ::= "while" <space> <condition> NEWLINE <block>
<if_stmt>           ::= "if" <space> <condition> NEWLINE <block>
                      | "if" <space> <condition> NEWLINE <block> "else"
                         NEWLINE <block>

<block>             ::= INDENT <statement_list> DEDENT

-- Function call
<function_call>     ::= <identifier> "()"
                      | <identifier> "(" <argument_list> ")"
```

```
<argument_list>     ::= <expression>
                      | <expression> <optional_space> "," <optional_space>
                          <argument_list>

<condition>         ::= <expression> <optional_space> <comparison_op> <
   optional_space> <expression>
                      | "not" <space> <condition>
                      | <function_call>
                      | <identifier>

<comparison_op>     ::= "==" | "!=" | "<" | ">" | "<=" | ">="

<expression>        ::= <term>
                      | <expression> <optional_space> "+" <optional_space>
                          <term>
                      | <expression> <optional_space> "-" <optional_space>
                          <term>

<term>              ::= <factor>
                      | <term> <optional_space> "*" <optional_space> <
                         factor>
                      | <term> <optional_space> "/" <optional_space> <
                         factor>

<factor>            ::= <literal>
                      | <identifier>
                      | <special_object>
                      | <function_call>
                      | "(" <optional_space> <expression> <optional_space>
                          ")"
                      | "-" <factor>

<special_object>    ::= <identifier>"."<identifier>

<literal>           ::= <number>
                      | <string>
```

```
                        | <boolean>

<boolean>             ::= "true" | "false"
<number>              ::= <digit> | <digit> <number>
<digit>               ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" |
   "8" | "9"
<string>              ::= "\"" <characters> "\""
<characters>          ::= ε | <character> <characters>
<character>           ::= <letter> | <special_char>

<identifier>          ::= <letter> | <letter> <id_tail>
<id_tail>             ::= <id_char> | <id_char> <id_tail>
<id_char>             ::= <letter> | <digit> | "_"

<letter>              ::= <lower> | <upper>

<lower>               ::= "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h" | "i
   " | "j" | "k" | "l" | "m"
                        | "n" | "o" | "p" | "q" | "r" | "s" | "t" | "u" | "v
                           " | "w" | "x" | "y" | "z"

<upper>               ::= "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H" | "I
   " | "J" | "K" | "L" | "M"
                        | "N" | "O" | "P" | "Q" | "R" | "S" | "T" | "U" | "V
                           " | "W" | "X" | "Y" | "Z"

<special_char>        ::= "!" | "\"" | "#" | "$" | "%" | "&" | "'" | "(" |
   ")"
                        | "*" | "+" | "," | "-" | "." | "/" | ":" | ";" |
                           "<"
                        | "=" | ">" | "?" | "@" | "[" | "\\" | "]" | "^" | "
                           _"
                        | "`" | "{" | "|" | "}" | "~" | <space>

<space>               ::= " "
<optional_space>      ::= <space> | ε
```

```
<comment>            ::= "--" <characters> NEWLINE
```